# Advanced Algorithms Deep Dive

A comprehensive exploration of Dijkstra's, Two Pointers, and Sliding Window techniques with visual logic and C++ implementation patterns.



PART 01

# Dijkstra's Algorithm

The Golden Standard for Shortest Paths in Weighted Graphs

# What is Dijkstra's Algorithm?



## Greedy Approach

The algorithm always selects the unvisited node with the absolute minimum known distance from the source to explore next.



## Edge Relaxation

Calculates the distance to a neighbor through the current node. If this path is shorter than the previously known one, update it.



## Complexity

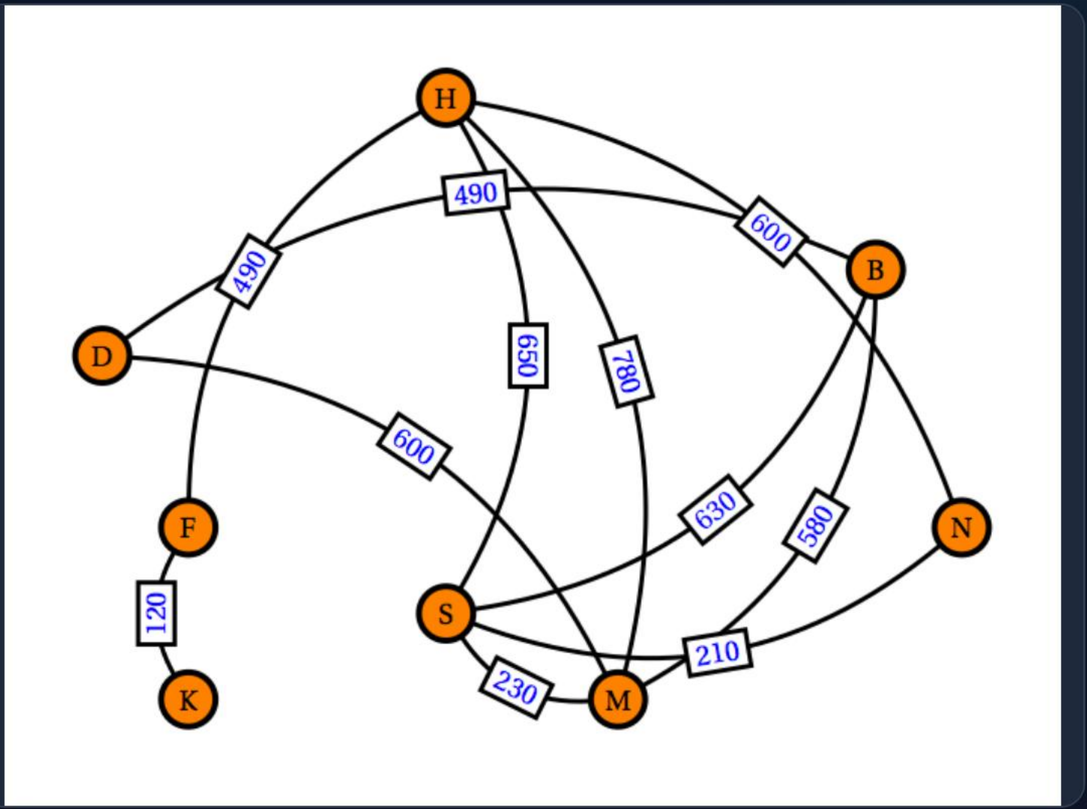
Using a Priority Queue (Min-Heap), the time complexity is `O((V+E) log V)`, making it efficient for large, sparse graphs.

# Dijkstra Example Walkthrough

## Step-by-Step Logic

Considering `Source Node A`, we find shortest distances to all reachable vertices in a weighted graph:

| Node | Shortest Dist. | Path Path |
|---|---|---|
| A | 0 | Source |
| B | 3 | A → C → B |
| C | 2 | A → C |
| D | 8 | A → C → B → D |
| F | 12 | ... → D → F |



# Execution Trace Analysis

### Step 1: Init

Push (0, A) to Heap.
Dist[A]=0, others=∞

### Step 2: Pop A

Relax neighbors. B=4, C=2.
Push both.

### Step 3: Pop C

C → B=3 < 4 (Update).
C → D=10, C → F=12.

### Step 4: Pop B

Relax B → D: 3+5=8 < 10
(Update D).

### Step 5: Pop D

D → E=10, D → F=14 > 12 (No update).

# C++ Implementation: Key Patterns

### </> Data Structures

```
priority_queue, greater> pq;
```

Uses a Min-Heap to ensure the smallest distance node is always at the top.

```
if (d > dist[u]) continue;
```

Handles "stale" entries efficiently without requiring a decrease-key operation.

### Core Logic

```
if (dist[u] + w < dist[v]) {
  dist[v] = dist[u] + w;
  pq.push({dist[v], v});
}
```

Updates distance only if a strictly shorter path is discovered via the current node.

# Real-World Use Cases

### GPS Navigation

Computing optimal driving routes in real-time road networks.

### Network Routing

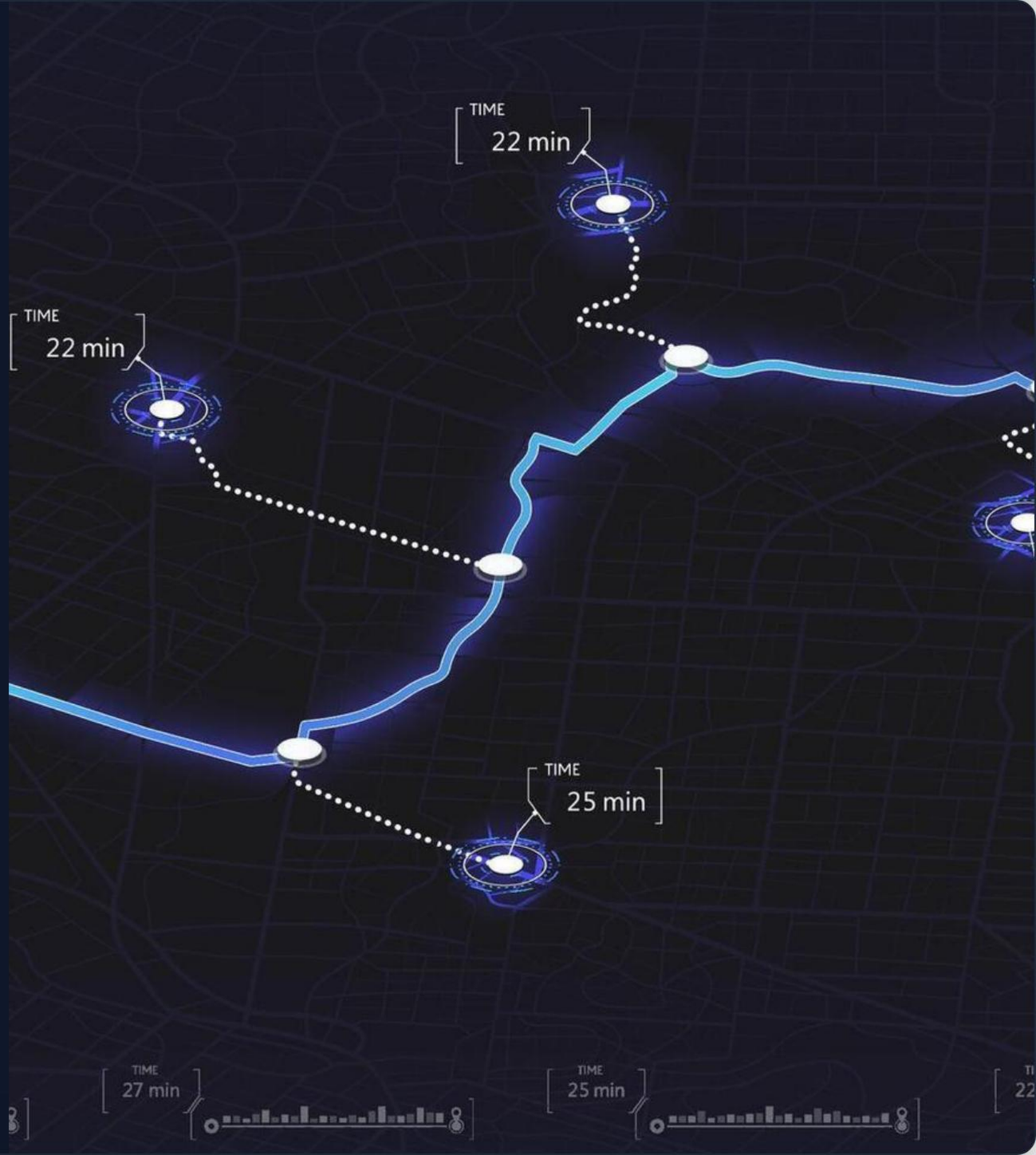
OSPF protocol finds optimal packet paths across routers.

### Game AI

NPCs use Dijkstra or A* to navigate complex game maps.

### Social Graphs

Calculating degrees of separation between users on LinkedIn.



PART 02

# Two Pointers Technique

Linear Array and String Processing Strategies

# Concepts and Patterns

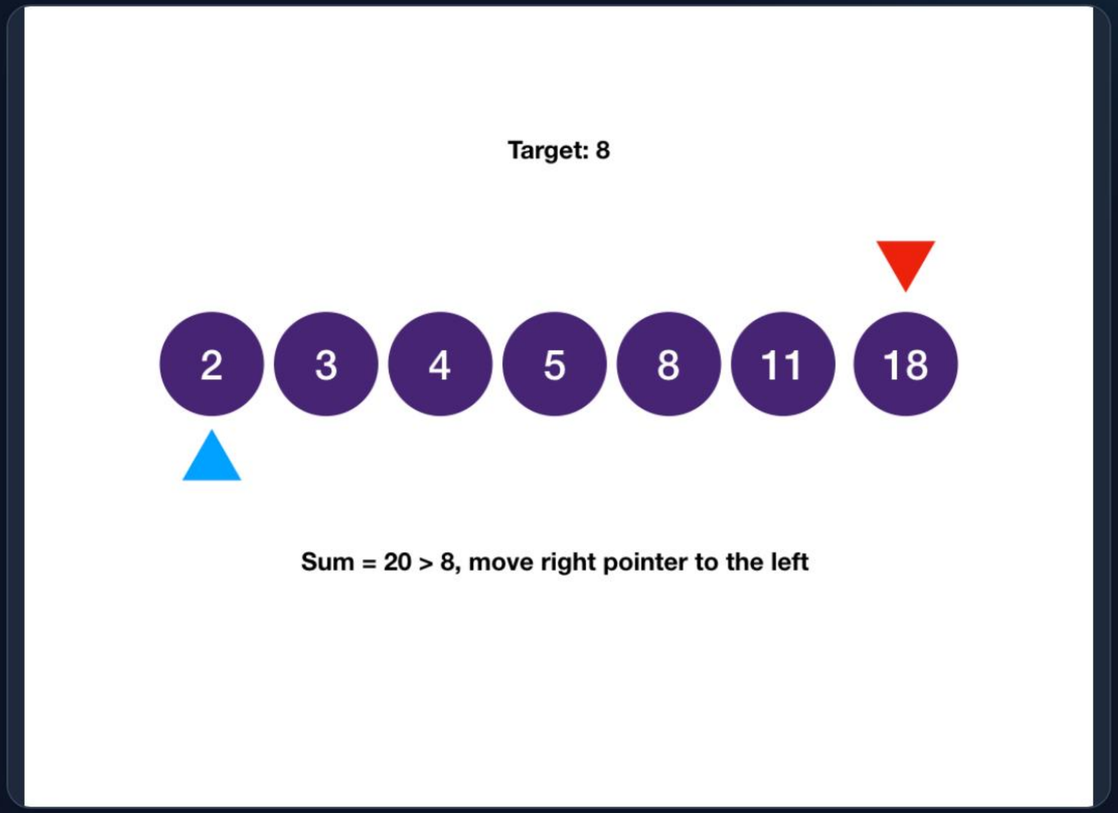
## Efficiency over Brute Force

Instead of Nested Loops `O(n²)`, we use two indices moving toward each other or in parallel to reach `O(n)`.

↔ **`Opposite Ends:`** Used for sorted arrays (Two Sum, Palindromes).

→ **`Same Direction:`** Used for partitioning or removing duplicates.

**`Merge Style:`** Comparing two separate sorted sequences.



PART 03

# Sliding Window Technique

Efficient Contiguous Subarray Management

# Fixed vs. Variable Windows

## Subarray Optimization

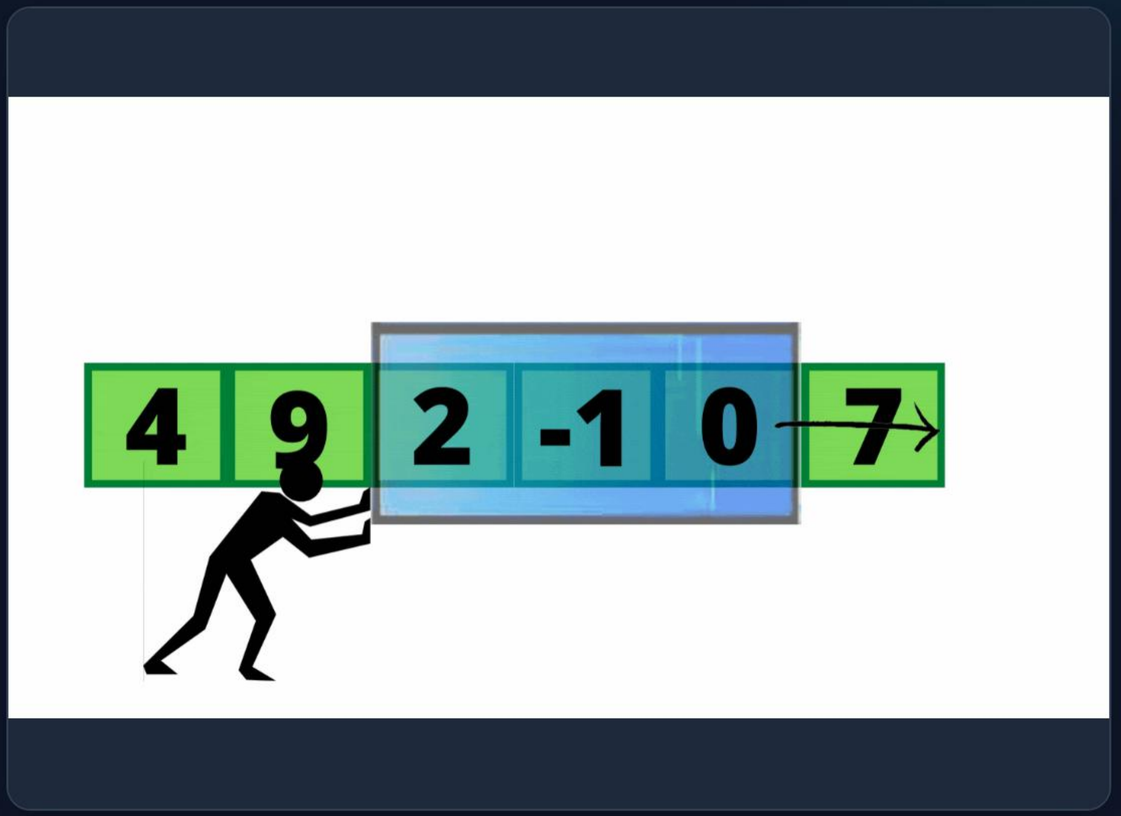
Maintains a contiguous window that expands or shrinks to compute properties efficiently.

### Fixed Size

Window size **k** stays constant. Slide by removing the tail and adding the head.

### Variable Size

Grows/shrinks based on conditions. Essential for "Longest substring without repeats".



# Patterns Cheatsheet & Summary

| Algorithm / Technique | Complexity | Data Context | Key Application |
| --- | --- | --- | --- |
| `Dijkstra's` | O((V+E) log V) | Weighted Graphs | Shortest path (GPS, Routing) |
| `Two Pointers` | O(N) | Sorted Arrays | Pair sums, duplicate removal |
| `Sliding Window` | O(N) | Subarrays / Substrings | Maximum sum, unique characters |

**"Mastering these patterns solves 60%+ of competitive programming problems."**

# Image Sources



https://media.istockphoto.com/id/1705828371/photo/emerging-digital-structure-growing-connection-lines-symbolizing-innovative-artificial.jpg?s=612x612&w=0&k=20&c=8PFxf8XDhsF0Q83PfXX6jQeLWD6lgwLmTroQOzVN5js=

Source: www.istockphoto.com

---



https://i.sstatic.net/KP59b.png

Source: tex.stackexchange.com

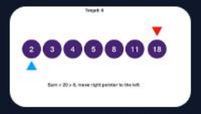
---



https://static.vecteezy.com/system/resources/previews/079/151/270/non_2x/isometric-gps-navigation-interface-with-glowing-route-path-and-location-markers-on-dark-city-map-illustration-vector.jpg

Source: www.vecteezy.com

---




https://assets.algo.monster/two_sum_sorted/two_sum_sorted.001.png

Source: algo.monster

---




https://i.sstatic.net/b5UIY.gif

Source: stackoverflow.com